QueueSmart – Assignment 3

Back-End Development & API Implementation

Team: Agah Duzenli, Dang (wabisabiz), Skyler Tran, Derek Lihou · October 2026

1. GitHub Repository

Repository: https://github.com/AgahDUZENLI/queuesmart (branch **main**)

The back end lives in `backend/` (source in `src/`, unit tests in `tests/`). The React front end from A2 is in `frontend/` and calls the back end through `frontend/src/api.js`.

Running the tests

```
cd backend
npm install
npm test # run all unit tests
npm run coverage # coverage report in coverage/lcov-report/index.html
```

The whole app can also be started with `docker compose up -d` (front end on port 5173, back end on port 4000). Demo logins are listed in `backend/README.md`.

Test results

Test suites	Tests	Statements	Branches	Functions	Lines
7 passed	109 passed	97.6%	91.0%	100%	99.2%

Coverage is above the 70–80% target. The tests cover business logic (queue ordering, serving, wait estimates, notifications, history) and validation rules (required fields, types, length limits, error responses).

Where each required module is

Required module	File(s)	Main endpoints
Authentication	auth.js, validation.js	POST /api/auth/register, /login, /logout; GET /api/auth/me; user vs admin roles
Service management	services.js	GET, POST /api/services; PUT, DELETE /api/services/:id
Queue management	queues.js	POST /api/queues/:id/join; DELETE /api/queues/tickets/:id; GET /api/queues/mine; POST /api/queues/:id/next, /complete, /no-show
Wait-time estimation	queues.js	Time left for the current visit + minutes per person × people ahead (average of recent visits)
Notifications	notifications.js	Created when a user joins and when they are close to being served; GET /api/notifications/me
History	history.js	GET /api/history/me, /api/history, /api/history/stats
Data (no database)	data.js, demo.js	In-memory collections, reset on restart

2. Backend Technologies

Technology	Used for
JavaScript on Node.js	Back-end language and runtime
Express 5	Web framework: REST API, routers, middleware
Jest + Supertest	Unit and API tests, coverage reports
Node crypto (scrypt)	Password hashing and session tokens
cors	Lets the React front end (another port) call the API
Docker Compose	Runs the front end and back end together

Why we chose them

- **One language for the whole team.** The A2 front end is React (JavaScript), so using JavaScript on the back end too meant everyone could work on either side without learning a second language.
- **Express is small and simple.** It has little setup, and each module is just a router file, which made it easy to split the six modules between team members.
- **Jest and Supertest work well together.** Jest runs the tests and measures coverage with no extra tools, and Supertest sends real HTTP requests to the app without starting a server.
- **No database needed yet.** Plain JavaScript arrays in memory are enough for A3, as allowed by the assignment.

How they support our QueueSmart design

- **Matches the A1 architecture.** Each module from A1 (authentication, services, queues, wait time, notifications, history) is its own file and router, mounted in `app.js`.
- **Roles are enforced in one place.** The `requireAuth` and `requireAdmin` middleware protect user and administrator routes.
- **Validation on every request.** Shared checks in `validation.js` enforce required fields, types and length limits, and return 400 errors with a message per field. The front end uses the same limits.
- **Connected to the A2 UI.** Every screen gets its data from the API (sign in, join and leave a queue, queue status, notifications, history, and the admin dashboard, services and queue pages).
- **Ready for A4.** All data goes through `data.js`, so it can be swapped for a real database in A4 without changing the routes.

3. Team Contributions

Group member name	What is your contribution?	Discussion notes
Agah Duzenli (GitHub: AgahDUZENLI)	Set up the A3 back end (Node, Express, Jest, Docker Compose). Built the Authentication module (register, login, user/admin roles, password hashing), Service management, places with join codes and shared validation, and connected sign-in and places to the front end. Connected notifications to the UI, added Call next / Done / No-show to the admin overview, made the wait estimate count down during a visit, and wrote the README.	Oct 17: shared the back-end setup and how to run it and the tests. Reviewed and merged pull requests from the team.
Dang (GitHub: wabisabiz)	Queue management and wait-time estimation in queues.js (join, leave, serve next, priority and arrival ordering, admin reprioritizing and reordering) with front-end integration. Admin pages, three extra demo user logins, and the "show all services" option on the admin page.	Oct 19: proposed splitting the six modules and took queue and wait-time. Oct 20: finished queue and wait-time. Oct 22: added demo users.
Skyler Tran (GitHub: skylertran19)	History module, front end and back end: history.js (user history, admin history, stats, status and limit filters) with tests, and connected the History screen (api.js, useUserQueue.js, removed the template rows).	Oct 19: took history. Oct 22: finished history (pull request #20).
Derek Lihou (GitHub: DerekLihou)	Notification module: notifications.js, which creates notifications when a user joins a queue and when they are close to being served (2 or fewer people ahead), with no duplicates per ticket. Hooked into the queue routes, with unit tests (notifications.test.js). Merged in pull request #22.	Oct 19: took notifications. Oct 22: finished notifications (pull request #22, merged).